Design and Construction of a Mobile Control Robot Arm on a Robotic Wheel

Oke Iyanuoluwa Enoch
Lecturer: Dr. P.K Olulope
B.Eng. Electrical and Information Engineering
College of Science and Engineering
Landmark University, Omu-aran, Nigeria
oke.iyanuoluwa@lmu.edu.ng, oke.iyanuoluwa12@gmail.com

***Abstract*—This paper presents the design, development, and implementation of an innovative mobile robotic system integrating wireless vision streaming, multi-axis articulated manipulation, and distributed microcontroller architecture. The system comprises a tank-wheeled mobile platform equipped with an ESP32-CAM module for real-time video surveillance and an Arduino Nano controlled six-degree-of-freedom robotic arm. The architecture employs a novel dual microcontroller approach with inter-device serial communication, enabling parallel processing of mobility control and manipulation tasks. A custom Android application provides intuitive remote operation through Bluetooth connectivity. The platform demonstrates significant advancement in low-cost robotics by achieving industrial grade functionality including live IP-based video streaming, LCD based operational feedback, and modular power management supporting solar charging integration. Experimental validation confirms successful wireless operation with real-time video latency under 200ms and precise manipulator positioning within $\pm 2^{\circ}$ accuracy. The system's scalable architecture and open-source implementation present opportunities for applications in agricultural automation, hazardous environment inspection, educational robotics, and research deployment scenarios requiring cost-effective mobile manipulation capabilities.**

***Index Terms*—Mobile robotics, ESP32 microcontroller, robotic manipulation, wireless control, embedded systems, IoT robotics, distributed computing, computer vision**

I. Introduction

The convergence of mobile robotics, wireless communication, and embedded systems has created unprecedented opportunities for autonomous systems in various industrial and research applications. Traditional robotic platforms often require significant capital investment and specialised infrastructure, limiting their accessibility to educational institutions and small-scale research initiatives. This work addresses this gap by presenting a comprehensive robotic system that achieves advanced functionality through innovative integration of commercially available components and custom software architecture.

The ZETABOT system represents a significant advancement in accessible robotics technology through several key innovations. First, the implementation of distributed processing architecture across ESP32-CAM and Arduino Nano platforms enables concurrent execution of computationally intensive vision streaming and real-time servo control. Second, the wireless communication framework supporting both Bluetooth control and WiFi based video transmission demonstrates practical IoT integration. Third, the modular power management system incorporating lithium battery arrays with solar charging capability addresses the critical challenge of field deployment sustainability.

Contemporary applications requiring mobile manipulation including precision agriculture, disaster response robotics, warehouse automation, and educational STEM programs demand platforms that balance functionality, cost effectiveness, and ease of deployment. This research contributes to that objective by demonstrating that sophisticated capabilities including live video feedback, articulated manipulation, and wireless operation can be achieved within accessible budget constraints without compromising operational performance.

A. Research Motivation and Problem Statement

The primary motivation for this research stems from the identified need for cost effective robotic platforms in developing regions and educational contexts where access to commercial robotic systems remains prohibitively expensive. Traditional industrial robotic arms and mobile platforms typically require investments exceeding $10,000, placing them beyond the reach of most educational institutions and small scale agricultural operations. Furthermore, existing low cost alternatives often compromise on critical capabilities such as wireless operation, vision systems, or manipulation dexterity.

Specific technical challenges addressed include: (1) achieving reliable wireless video streaming with minimal latency using constrained embedded hardware, (2) coordinating multiple servo actuators through resource limited microcontrollers, (3) implementing robust Bluetooth communication protocols for real-time control, and (4) developing user friendly mobile interfaces that require minimal technical expertise. The solution presented herein demonstrates that these challenges can be overcome through innovative architectural decisions and careful component selection.

B. Research Objectives

The specific objectives of this research are:

- Design and implement a distributed microcontroller architecture optimising task allocation between mobility control and manipulation systems

- Develop a wireless communication framework supporting concurrent Bluetooth control and WiFi video streaming with latency suitable for real-time operation
- Engineer a six-degree-of-freedom robotic arm achieving precise positioning control through serial communication protocol
- Create an intuitive Android mobile application providing comprehensive system control and operational feedback
- Validate system performance through experimental testing of mobility, manipulation accuracy, communication reliability, and power consumption characteristics

II. Literature Review and Related Work

A. Evolution of Mobile Robotic Systems

Mobile robotics has undergone substantial evolution from early industrial applications to contemporary IoT enabled autonomous systems. Siegwart et al. established foundational principles of autonomous mobile robot design, emphasising the integration of sensing, actuation, and control within constrained computational environments [31]. Traditional mobile robotic platforms historically relied on centralised processing architectures employing single board computers or industrial controllers, limiting accessibility due to cost and complexity considerations.

Recent advancements in microcontroller technology, particularly the emergence of wireless enabled platforms such as the ESP32 family, have democratised mobile robotics development. These platforms integrate computational capability, wireless connectivity, and peripheral interfaces within compact, energy efficient packages suitable for battery operated mobile applications. The dual-core architecture of ESP32 processors enables concurrent task execution, addressing traditional bottlenecks in wireless communication and real-time control applications.

B. Distributed Control Architectures in Robotics

Distributed control architectures represent a fundamental approach to managing complexity in robotic systems through task partitioning across multiple processing units. Jahn et al. [15] present comprehensive concepts for modular distributed architectures in robotic systems, introducing the Operator Controller Module framework that hierarchically organises system components into distinct functional layers. Their work demonstrates that distributed architectures enable scalable system design, facilitate fault isolation, and optimise resource utilisation across heterogeneous processing platforms.

The advantages of distributed processing extend beyond mere computational load balancing. Research in industrial robotics demonstrates that segregating real-time control functions from higher level planning and communication tasks improves system responsiveness and determinism. This architectural approach aligns with the hierarchical control paradigm prevalent in advanced robotic systems, where lower level controllers manage immediate sensor actuator interactions while higher level processors handle strategic decision making and human interface functions.

Implementation of distributed architectures traditionally required specialised middleware and complex inter process communication frameworks. However, lightweight serial communication protocols offer pragmatic alternatives for resource constrained embedded systems. The simplicity of UART based command protocols enables reliable inter controller communication with minimal overhead, particularly suitable for applications not requiring high bandwidth data exchange between processing nodes.

C. ESP32 Based Robotic Applications

The ESP32 microcontroller family has gained substantial adoption in mobile robotics applications due to its combination of processing capability, wireless connectivity, and cost effectiveness. Research demonstrates diverse implementations ranging from autonomous navigation systems to teleoperated inspection platforms. Studies have explored voice controlled mobile robotics utilising ESP32's processing capabilities for command recognition and wireless control [19].

The ESP32 CAM variant, integrating camera capabilities with wireless communication, has enabled surveillance and monitoring applications. Mobile surveillance platforms employing ESP32 CAM for real-time video streaming over WiFi networks demonstrate the feasibility of vision based robotics within embedded system constraints [17]. These implementations typically achieve frame rates of 15-20 fps at VGA resolution, adequate for human operated teleoperation and basic computer vision tasks.

Integration of ESP32 platforms with Android based mobile interfaces represents a common architectural pattern in IoT robotics. Bluetooth Serial Profile provides straightforward command transmission mechanisms, enabling rapid prototyping of control applications without complex network configuration [20]. Implementations demonstrate reliable operation at distances up to 10 meters with latency suitable for human-in-the-loop control applications, validating this approach for educational and research deployments.

D. Robotic Manipulator Design and Control

Robotic manipulator design encompasses mechanical configuration, actuator selection, and control algorithm implementation. Classical manipulator theory emphasises kinematic analysis for position and orientation control, with industrial applications typically requiring precise trajectory planning and force control capabilities. However, educational and demonstration platforms often employ simplified control schemes prioritising ease of implementation over absolute positioning accuracy.

Low cost manipulator implementations commonly utilise hobby servo motors for actuation, trading precision for affordability and simplicity. Standard hobby servos provide integrated position control through PWM signal interpretation, abstracting motor driver complexity and enabling straightforward microcontroller interfacing. Multi DOF arms employing serial chains of servo actuators can achieve workspace sufficient

for pick-and-place operations and object manipulation tasks relevant to educational demonstrations.

Recent research explores ESP32 based manipulator control for educational applications, demonstrating WebSocket and Bluetooth control interfaces. These implementations validate microcontroller based servo control for multi joint arms, with studies reporting successful operation of 4-7 DOF systems using distributed control architectures. The integration of real-time feedback displays enhances user experience and facilitates debugging during development, representing best practices in embedded robotics design.

E. Research Gap and Contribution

While existing research demonstrates individual components mobile platforms with ESP32 control, robotic arms with microcontroller actuation, and wireless video streaming systems comprehensive integration of these capabilities within unified, cost-effective platforms remains limited. Most documented systems implement either mobility or manipulation, but not both with independent control architectures optimised for concurrent operation.

This work advances the state of art through several distinct contributions. First, the implementation of true distributed processing across heterogeneous microcontroller platforms (ESP32-CAM and Arduino Nano) demonstrates practical task allocation strategies maximising individual platform strengths. Second, the integration of multiple wireless protocols (WiFi for video, Bluetooth for control) within resource constrained hardware validates concurrent communication feasibility. Third, the modular power architecture with solar charging capability addresses deployment sustainability considerations often overlooked in research prototypes. Finally, comprehensive experimental validation across multiple performance dimensions provides quantitative assessment facilitating comparison with alternative approaches and informing future development efforts.

III. System Architecture and Design

A. Hardware Architecture Overview

The ZETABOT hardware architecture employs a distributed processing paradigm partitioning computational responsibilities across two specialised microcontroller units. The ESP32-CAM module serves as the primary system controller, managing wireless communications (Bluetooth and WiFi), motor driver coordination for the mobile platform, and video acquisition/streaming. The Arduino Nano operates as a dedicated manipulator controller, executing real-time servo positioning calculations and managing the LCD user interface. This architectural decision addresses the computational limitations of individual microcontrollers while maintaining system simplicity and cost effectiveness.

Inter controller communication utilises asynchronous serial protocol at 9600 baud, transmitting encoded command integers representing specific manipulator actions. This lightweight communication scheme minimises latency and processing overhead. The mobile platform employs tank style differential drive configuration with L298N dual H-bridge motor drivers, enabling omnidirectional mobility including forward/reverse translation and in place rotation.

B. Microcontroller Selection and Specifications

1) ESP32-CAM Module: The ESP32-CAM represents a critical component selection enabling advanced functionality within budget constraints. The ESP32-CAM module is based on the ESP32 dual-core Tensilica LX6 processor operating at up to 240 MHz, and integrates an OV2640 2-megapixel camera sensor, 520 KB of SRAM, and dual mode wireless connectivity (802.11 b/g/n Wi-Fi and Bluetooth). The dual-core architecture proves essential for concurrent execution of video encoding and wireless transmission tasks without compromising control responsiveness.

Key technical specifications include 4MB flash memory supporting firmware storage and temporary video buffering, multiple GPIO pins (though limited compared to standard ESP32 modules due to camera interface requirements), and onboard antenna for wireless communication. The module's integrated voltage regulator accepts 5V input, simplifying power distribution design. Video streaming capability achieving 15-20 fps at VGA resolution (640×480) through HTTP server implementation represents significant functionality at approximately $10 component cost.

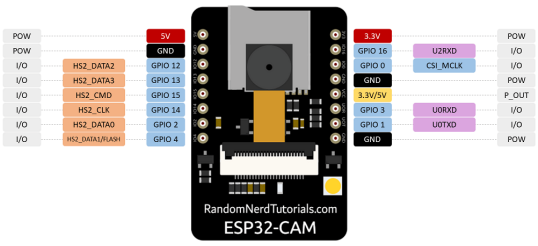

Fig. 1: Layout of ESP32-CAM module

2) Arduino Nano: The Arduino Nano, based on the ATmega328P microcontroller (16MHz, 32KB flash, 2KB SRAM), serves as the dedicated manipulator control unit. Its selection addresses specific advantages for servo control applications: hardware PWM capabilities on multiple pins enabling simultaneous servo actuation, extensive library support for servo control abstraction, and compact form factor (18×45mm) facilitating integration into the manipulator assembly. The Nano's 19 digital I/O pins provide sufficient connectivity for six servo motors, I2C LCD interface, and serial communication with the ESP32-CAM. Power consumption characteristics (19mA active current) enable extended operation from the 7.4V lithium battery pack dedicated to the manipulator subsystem.

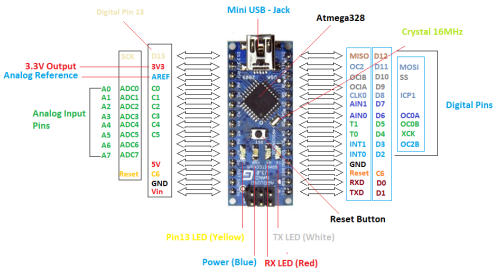

Fig. 2: Layout of Arduino Nano

C. Six-Degree-of-Freedom Manipulator Design

The robotic arm adopts a six-degree-of-freedom (6-DOF) anthropomorphic configuration consisting of base rotation (waist), shoulder articulation, elbow flexion, wrist pitch, wrist roll, and gripper actuation. This kinematic structure enables operation within a hemispherical workspace while maintaining computationally efficient inverse kinematics suitable for microcontroller-based implementation.

Each joint is actuated using an MG996R servo motor, which operates using pulse width modulation (PWM) control signals. The MG996R provides high torque output and reliable angular positioning, making it suitable for robotic manipulation tasks requiring stable load handling and repeatable motion.

The structural components are fabricated from laser cut acrylic panels to achieve a balance between mechanical rigidity and low overall weight. The arm provides an approximate base-to-gripper reach of 30 cm and supports payloads up to 100 g, which is sufficient for demonstration and light-duty manipulation applications.

The control system employs incremental joint positioning rather than absolute Cartesian coordinate targeting, thereby simplifying software complexity while maintaining adequate operational precision. Each servo is driven in discrete angular steps, and real-time system status is displayed via an LCD interface to provide user feedback and operational verification.

D. Tank Tracked Mobile Platform Configuration

The mobile base employs tracked drive configuration with independent left and right motors enabling differential steering. This design provides superior traction on irregular surfaces compared to wheeled alternatives, critical for field deployment scenarios. Two geared DC motors (12V, 150 RPM, 3 kg-cm torque) drive the tracks through 3D printed sprocket assemblies. The L298N H-bridge motor driver manages bidirectional control and enables PWM speed regulation, though the current implementation operates at fixed speed due to adequate performance for demonstration purposes. Platform dimensions of 175 mm $\times$ 180 mm $\times$ 85 mm (L $\times$ W $\times$ H, arm excluded), provide stable base for manipulator mounting while maintaining manoeuvrability in constrained environments. Total system weight of approximately 1.5kg enables operation duration exceeding 2 hours on battery capacity described subsequently.

E. Modular Power Management Architecture

The power architecture implements separate battery systems for mobility and manipulation subsystems, addressing distinct voltage and current requirements while simplifying fault isolation. The mobile platform operates from three series connected 18650 lithium ion cells providing 11.1V nominal voltage with 2500mAh capacity. An XL6019 DC-DC boost converter steps this voltage to 12V for motor driver operation, compensating for battery discharge characteristics. The manipulator subsystem utilises two 18650 cells (7.4V, 2500mAh) directly powering servos, with onboard 5V regulation for Arduino Nano and LCD display.

A 18650 3S charge management module enables simultaneous charging of the mobility battery pack while the manipulator pack charges independently. Integration of 5.5V, 70mA solar panels provides trickle charging capability extending deployment duration in outdoor scenarios, though primary recharging occurs through standard USB power adapters. This modular approach enables battery replacement without system shutdown and facilitates future capacity expansion. Measured power consumption averages 2.1A for the mobile platform during continuous operation and 0.8A for the manipulator during active manipulation, yielding operational duration of approximately 2.5 hours under typical duty cycles.

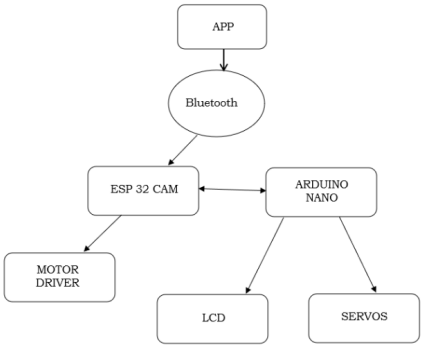

Fig. 3: Block diagram of the ZETABOT system architecture showing Bluetooth based command flow from the mobile application to the ESP32-CAM, distributed control with the Arduino Nano, and actuation through motor driver, LCD, and servo subsystems.

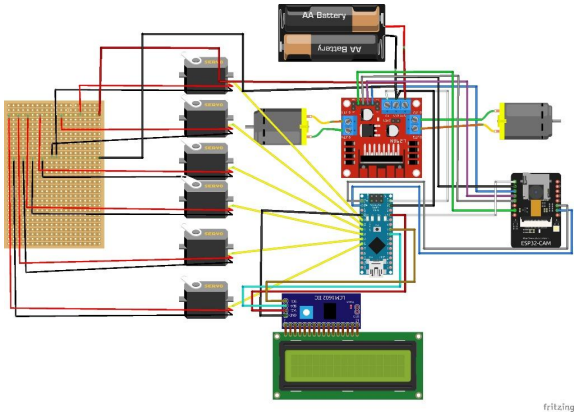

Fig. 4: Pictorial wiring diagram of the ZETABOT system showing physical interconnections between ESP32-CAM, Arduino Nano, motor driver, servo motors, LCD module, and power supply components.

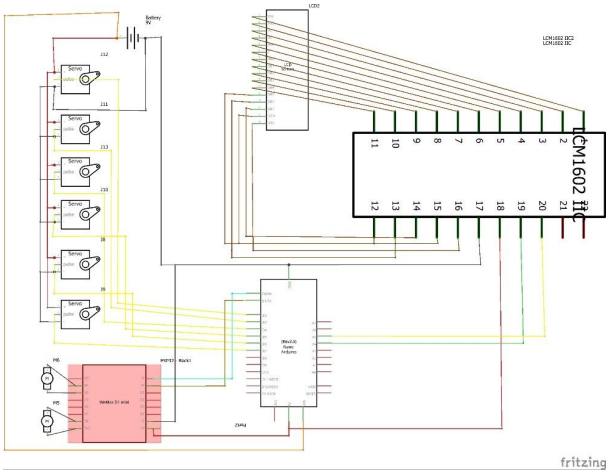

Fig. 5: Electrical schematic of the ZETABOT control system illustrating signal routing, power distribution, and peripheral interfacing.

F. Physical Dimensions and Mechanical Specifications

The physical dimensions of the ZETABOT platform are summarised as follows:

- Arm height: 460 mm
- Tank base height: 85 mm
- Tank base length: 175 mm
- Tank base width: 180 mm

These dimensions provide a compact footprint suitable for operation in confined indoor environments while maintaining sufficient manipulator reach for object handling and inspection tasks. The low profile tracked base contributes to platform stability during arm articulation and motion over uneven surfaces.

IV. Methodology and System Implementation

A. Development Workflow

This project followed a structured engineering development methodology consisting of requirement analysis, hardware selection, system modelling, circuit design, firmware implementation, mobile application development, integration testing, and experimental validation. Figure 6 illustrates the overall development workflow adopted for the ZETABOT platform.

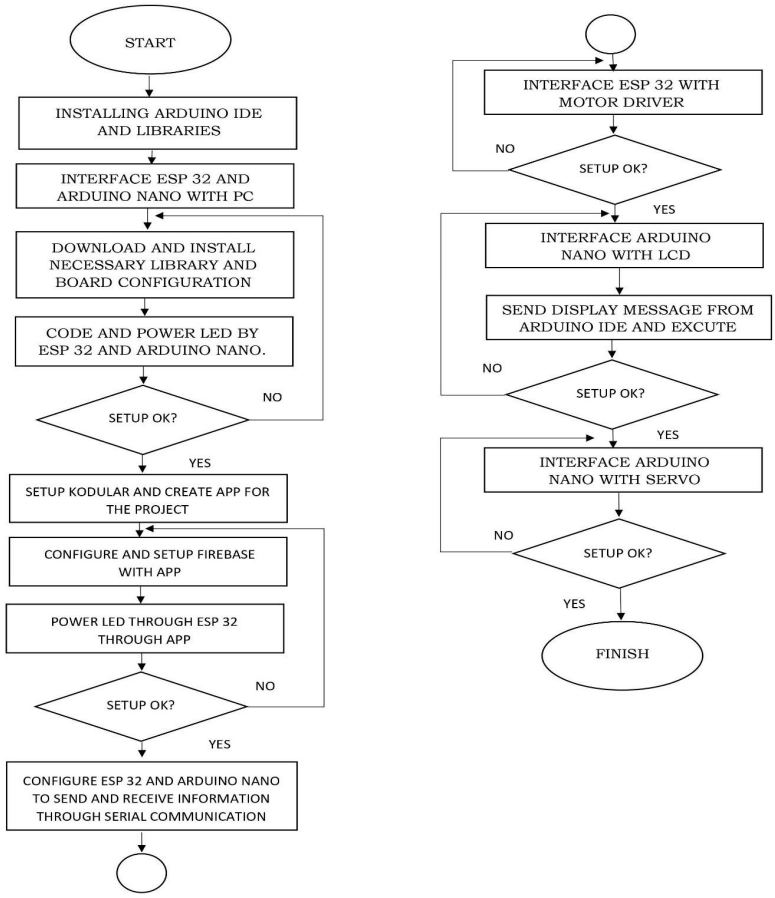

Fig. 6: ZETABOT system development methodology workflow.

B. Hardware Assembly Procedure

The hardware assembly of the ZETABOT platform was carried out using a modular and subsystem oriented approach to ensure reliability, ease of debugging, and scalability. The complete system was divided into four principal subsystems: (i) processing and communication unit, (ii) mobile platform and motor drive unit, (iii) robotic manipulator unit, and (iv) power management unit. Each subsystem was independently assembled and tested before full system integration.

1) ESP32-CAM Programming and Configuration Setup: The ESP32-CAM module was designated as the primary controller for wireless communication, motor control, and real-time video streaming. Initial configuration was performed using an external USB-to-TTL (FTDI) converter module due to the absence of a native USB interface on the ESP32-CAM board.

Electrical connections were established as follows:

- ESP32-CAM 5V $\rightarrow$ FTDI 5V

- ESP32-CAM GND → FTDI GND
- ESP32-CAM U0R (RX) → FTDI TX
- ESP32-CAM U0T (TX) → FTDI RX
- ESP32-CAM GPIO0 → GND (temporary, to enable flashing mode)

The Arduino IDE was configured with the Espressif ESP32 board support package via the official package index repository. The AI Thinker ESP32-CAM board profile was selected during compilation and upload. After successful firmware flashing, the GPIO0-GND connection was removed and the module rebooted into execution mode.

The ESP32-CAM was physically mounted on the upper chassis layer to minimise electromagnetic interference from the motor driver circuitry and to provide an unobstructed field of view for the OV2640 camera module.

2) Arduino Nano Installation and Peripheral Interface: The Arduino Nano was employed as a dedicated real-time controller for the robotic arm and liquid crystal display (LCD) subsystem. The board was soldered onto a perforated vero board to improve mechanical stability and vibration resistance.

The following connections were established:

- Serial communication lines:
  - Arduino RX → ESP32-CAM TX (via logic-level compatible interface)
  - Arduino TX → ESP32-CAM RX
- Power supply:
  - VIN → 7.4 V battery pack (regulated onboard)
  - GND → system common ground

This separation of processing tasks allowed the ESP32-CAM to focus on communication and mobility control while the Arduino Nano handled deterministic servo timing and display operations, preventing resource contention during peak loads.

3) Motor Driver and Mobile Platform Integration: A dual H-bridge L298N motor driver module was used to control the tracked mobile base. Two 12 V geared DC motors were mounted symmetrically on the left and right sides of the tank chassis using metal brackets to ensure alignment and minimise mechanical backlash.

Motor wiring was configured as:

- Motor A → OUT1 and OUT2 terminals
- Motor B → OUT3 and OUT4 terminals

Control signals were mapped to ESP32-CAM GPIO pins:

- IN1, IN2 → Left motor direction control
- IN3, IN4 → Right motor direction control

The motor driver supply voltage (VMS) was connected directly to the 11.1 V battery pack via the boost converter output. Logic voltage (5 V) was derived from the onboard regulator.

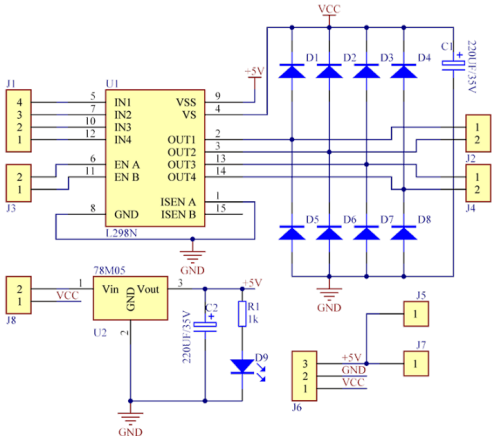

Fig. 7: Schematic diagram for motor drive module

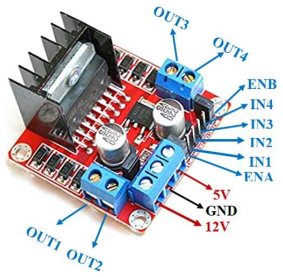

Fig. 8: Layout of L298N Module

Rubberised tank tracks were installed over the sprockets and tensioned to prevent slippage during acceleration and turning. Static and dynamic rotation tests were conducted to verify correct polarity and direction mapping before integration with the robotic arm.

4) Robotic Arm Mechanical Assembly and Servo Installation: The manipulator structure was assembled using laser cut acrylic components arranged in a serial kinematic chain comprising base rotation, shoulder, elbow, wrist pitch, wrist rotation, and gripper actuation joints.

Each joint was actuated using an MG996R high-torque servo motor secured with M2 bolts and nylon spacers to reduce vibration transfer. Servo horn alignment was calibrated manually to establish a neutral reference position (90°) before mechanical fastening.

Servo wiring was standardised as:

- Red → +5 V supply
- Brown/Black → Ground
- Yellow/Orange → PWM control signal

The servo power rail was isolated from the logic supply using a dedicated 7.4 V battery pack to prevent voltage sag during high-torque movements.

Mechanical load testing confirmed stable articulation under a payload of approximately 100 g at full arm extension.

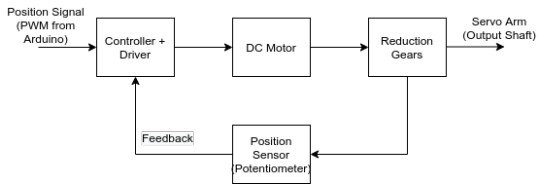

Fig. 9: Schematic diagram for MG996R Servo Motor

Fig. 10: Layout of MG996R Servo Motor

5) I2C LCD Module Integration: A 16×2 I2C LCD module was installed on the frontal panel of the platform to provide system status and operator feedback.

Connections were implemented as:

- LCD VCC → Arduino 5 V
- LCD GND → Arduino GND
- LCD SDA → Arduino A4
- LCD SCL → Arduino A5

The I2C backpack reduced wiring complexity from 16 conductors to four, improving reliability and ease of maintenance. The LCD was secured using vibration damping adhesive mounts.

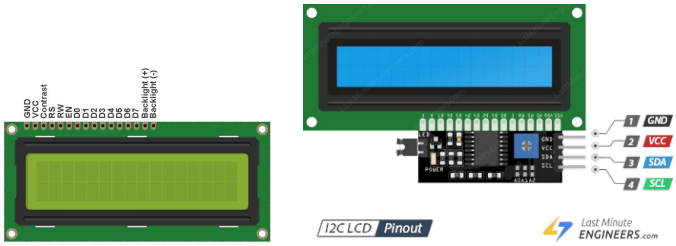

Fig. 11: Layout of 16×2 LCD with I2C Module

6) Power System Assembly: The power architecture employed dual battery packs:

- Mobility pack: Three 18650 Li-ion cells in series (11.1 V nominal)
- Manipulator pack: Two 18650 Li-ion cells in series (7.4 V nominal)

A 3S lithium battery protection and charging module provided over-charge, over-discharge, and short-circuit protection. An XL6019 DC-DC boost converter regulated voltage supplied to the motor driver under varying load conditions.

A master toggle switch was installed on the main supply line for emergency shutdown. Ground lines from all subsystems were tied to a common reference to prevent floating potentials and communication instability.

Optional 5.5 V solar panels were connected through a charge controller to enable trickle charging during outdoor deployment.

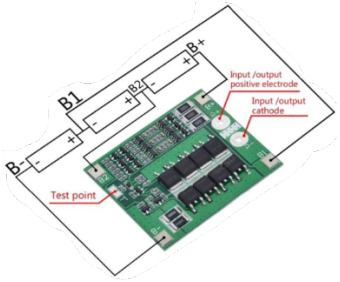

Fig. 12: Layout of 18650 15A 3S Charger Module

7) Subsystem Validation Prior to Full Integration: Before full system coupling, each subsystem underwent standalone testing:

- ESP32-CAM: Bluetooth discovery, WiFi connection, camera stream verification
- Arduino Nano: Servo sweep tests, LCD output validation
- Motor driver: Forward, reverse, and pivot rotation tests
- Power system: Voltage stability and current draw measurement

Only after successful validation were the subsystems mechanically and electrically integrated into the final ZETABOT platform.

C. Firmware Development Methodology

The ZETABOT platform employs a distributed firmware architecture in which the ESP32-CAM and Arduino Nano execute independent but coordinated control programs. This separation enables real-time motor and vision processing to be handled by the ESP32-CAM while deterministic servo control and user feedback display are managed by the Arduino Nano. The two controllers communicate through a lightweight asynchronous serial protocol, allowing reliable command exchange with minimal computational overhead.

1) ESP32-CAM Firmware Implementation: The ESP32-CAM firmware was developed using the Arduino framework and structured into four functional layers: Bluetooth communication handling, command parsing and dispatch, motor control via GPIO mapping, and WiFi based camera streaming using an embedded HTTP server.

a) Bluetooth Communication Using Bluetooth Serial Library: Wireless command reception is implemented using the built-in Bluetooth Serial library provided by the ESP32 SDK. This library enables the microcontroller to operate as a Bluetooth Classic Serial Port Profile (SPP) device, allowing direct pairing with Android smartphones without additional middleware or network configuration.

During system initialisation, the ESP32-CAM advertises itself using a unique device identifier (“ZETABOT”). Once paired, the microcontroller listens continuously on the Bluetooth serial buffer for incoming ASCII encoded control commands transmitted by the mobile application.

This approach provides:

- Low-latency communication suitable for human-in-the-loop control
- Minimal memory footprint
- High compatibility with Android devices
- Simple integration with Arduino based firmware environments

b) Command Parser Design: Incoming Bluetooth data is processed using a character buffer accumulation strategy. Characters are appended to a temporary command string until a termination symbol ("#") is detected. This delimiter indicates completion of a command packet and triggers parsing logic.

The parser then compares the decoded command against predefined instruction tokens representing:

- Mobile platform motion commands (forward, reverse, left, right, stop)
- Lighting control
- Robotic arm joint movement commands

Each recognised command is translated into either direct GPIO actuation (for motors and lighting) or encoded serial messages forwarded to the Arduino Nano for manipulator control.

This string based protocol simplifies debugging and ensures resilience against partial packet reception or transmission noise.

c) Motor GPIO Mapping Strategy: The ESP32-CAM controls the L298N dual H-bridge motor driver using four digital GPIO outputs. Each output corresponds to one direction control pin of the motor driver:

- IN1, IN2 $\rightarrow$ Left track motor direction
- IN3, IN4 $\rightarrow$ Right track motor direction

Logical combinations of HIGH and LOW signals implement the following motion primitives:

TABLE I: Motor GPIO Mapping for Motion Control

Motion Mode	**IN1**	**IN2**	**IN3**	**IN4**
Forward	1	0	1	0
Reverse	0	1	0	1
Rotate Left	0	1	1	0
Rotate Right	1	0	0	1
Stop	0	0	0	0

This direct mapping minimises processing overhead and ensures rapid response to operator input.

d) HTTP Camera Server Implementation: Real-time video streaming is achieved using the official ESP32 CameraWebServer example provided within the ESP32 Arduino SDK. The firmware initialises the OV2640 camera module and configures an embedded HTTP server operating on port 80.

Key characteristics include:

- MJPEG stream format
- VGA resolution (640 $\times$ 480)
- Frame rate of 15-20 fps
- Dynamic IP allocation via DHCP

Upon startup, the assigned IP address is transmitted over the serial monitor for client access. Any device connected to the same WiFi network can access the live stream through a standard web browser or video client.

This implementation enables visual navigation and manipulation feedback without requiring additional hardware or cloud infrastructure.

2) Arduino Nano Firmware Implementation: The Arduino Nano firmware was developed to provide deterministic control of the robotic arm and real-time system feedback through the LCD module. The firmware architecture consists of four primary components: serial command interpretation, servo actuation, LCD initialisation and messaging, and safety boundary enforcement.

a) Serial Communication Protocol: The Arduino Nano communicates with the ESP32-CAM via UART at 9600 baud. Commands are transmitted as integer values representing discrete motion operations for individual joints.

This numeric protocol was selected to:

- Reduce transmission overhead
- Simplify parsing logic
- Improve reliability under noisy electrical conditions

Upon reception of a valid command integer, the microcontroller triggers the corresponding servo motion routine and updates the LCD status display.

b) Servo Control Using Arduino Servo Library: The standard Servo.h library is used to generate PWM signals required for angular positioning of each joint actuator. Six servo objects are instantiated, corresponding to:

1) Base (waist)
2) Shoulder
3) Elbow
4) Wrist pitch
5) Wrist rotation
6) Gripper

Servo positions are stored in software variables to maintain state awareness and support incremental movement. Each command adjusts the servo position by a fixed angular step (typically 5-10 degrees) to allow smooth and controlled motion.

Mechanical safety limits are enforced in software to prevent commands exceeding the physical joint range.

c) LCD Initialisation and User Feedback: A 16$\times$2 LCD module connected via I2C is initialised during startup using the LiquidCrystal_I2C library. The display provides real-time system feedback including:

- System boot confirmation
- Bluetooth and WiFi connection status
- Active operation mode
- User identification message

LCD feedback improves operator confidence and simplifies debugging during development and field deployment.

d) Command Mapping Table: The Arduino firmware maintains a fixed mapping between received command integers and physical joint actions, as summarised below:

TABLE II: Command Code Mapping for Robotic Arm Control

Command Code	Joint	Action
31 / 32	Base	Rotate right/left
33 / 34	Shoulder	Up / Down
35 / 36	Elbow	Up / Down
37 / 38	Wrist pitch	Up / Down
39 / 40	Wrist rotate	Right / Left
41 / 42	Gripper	Open / Close

This abstraction decouples high-level user interaction from low-level actuation logic, enabling future extension without modifying the mobile application protocol.

The complete firmware source code is publicly available in the project GitHub repository for reproducibility and community development.

D. Android Application Development Using Kodular

The Android control application was developed using Kodular, a block based visual programming environment derived from MIT App Inventor. This approach enables rapid interface design and reliable Bluetooth integration without extensive native Android programming overhead.

1) User Interface Design: The application interface was designed to prioritise simplicity and operational clarity. The layout consists of:

- Directional control pad for mobile platform navigation
- Individual joint control buttons for robotic arm manipulation
- Bluetooth connection management controls
- System status indicators

Large touch targets and high contrast icons were employed to ensure usability under outdoor lighting conditions and on small screen devices.

2) Bluetooth SPP Communication: The application uses Bluetooth Classic Serial Port Profile (SPP) to communicate with the ESP32-CAM. Upon launch, the application scans for nearby devices, identifies the ZETABOT module, and establishes a persistent serial connection.

Connection status is continuously monitored, and automatic reconnection is attempted if communication is interrupted.

3) Command Encoding Strategy: Each button press generates a predefined ASCII command sequence terminated by the "#" delimiter. Commands are transmitted immediately without buffering to minimise control latency.

This stateless protocol ensures:

- Fast response
- Minimal processing requirements on the microcontroller
- Robust recovery after temporary connection loss

4) Testing Using Kodular Companion: During development, the Kodular Companion application was used to test interface logic in real time on physical Android devices. This enabled iterative debugging of:

- Button event handling
- Bluetooth pairing logic
- Command transmission reliability
- Interface layout scaling across different screen sizes

The final application package (APK) was exported and installed on multiple test devices to validate cross device compatibility.

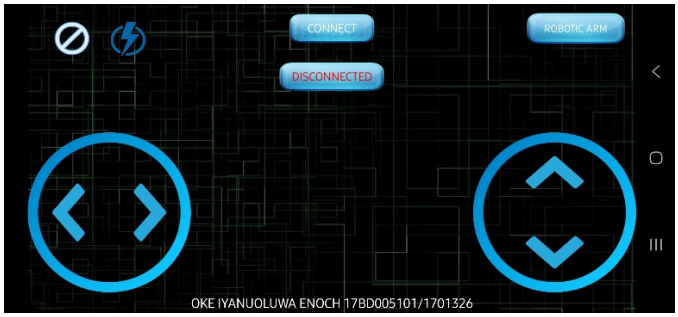

Fig. 13: Zetabot App development interface

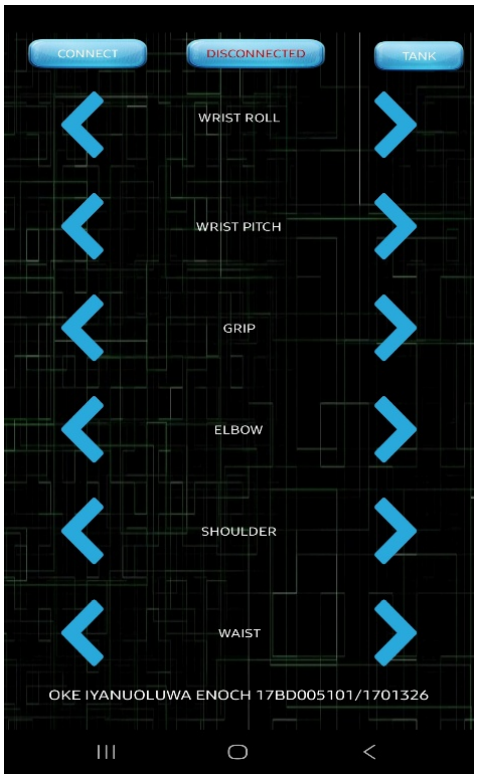

Fig. 14: Screenshot of the Zetabot App

E. System Integration and Testing Procedure

System integration followed a structured incremental methodology to minimise fault propagation and simplify troubleshooting.

1) Step-Wise Integration Process: The integration sequence proceeded as follows:

1) Standalone validation of ESP32-CAM Bluetooth and WiFi functionality
2) Standalone validation of Arduino Nano servo control and LCD output
3) Motor driver and mobile base integration with ESP32-CAM

4) Serial communication linkage between both microcontrollers
5) Power subsystem coupling
6) Mechanical assembly of full platform

Each stage was verified before proceeding to the next.

2) Unit Testing of Subsystems: Subsystem level testing included:

- Motor direction and braking verification
- Servo angular accuracy testing
- LCD message rendering
- Bluetooth command reliability
- Camera stream stability
- Battery voltage regulation under load

Faults were isolated and corrected at this stage to prevent cascading failures during full system operation.

3) End-to-End Functional Testing: After complete assembly, the system underwent end-to-end testing involving simultaneous:

- Mobile navigation
- Robotic arm manipulation
- Live video streaming
- LCD feedback monitoring

Latency, command accuracy, and system stability were measured during continuous operation cycles exceeding 30 minutes.

4) Field Testing: Field testing was conducted in both indoor laboratory environments and outdoor open spaces. The robot was evaluated on tiled floors, concrete surfaces, and mild uneven terrain to assess traction, manoeuvrability, wireless range, and visual feedback clarity.

Environmental testing confirmed stable operation under moderate sunlight conditions and distances up to 8-10 meters for Bluetooth control, validating the platform's suitability for educational, agricultural, and inspection scenarios.

Fig. 15: Fully assembled ZETABOT mobile robotic platform showing tracked base, six-degree-of-freedom manipulator, onboard ESP32-CAM vision module, LCD feedback unit, and power subsystem prior to field testing.

V. Experimental Validation and Performance Analysis

A. Testing Methodology and Experimental Setup

Comprehensive testing evaluated system performance across multiple operational dimensions: wireless communication reliability, video streaming characteristics, manipulator positioning accuracy, mobile platform manoeuvrability, power consumption profiles, and overall system integration. Testing occurred in controlled laboratory environments and representative deployment scenarios including indoor navigation and outdoor operation under direct sunlight conditions. WiFi connectivity utilised institutional network infrastructure providing stable baseline for video streaming assessment, while Bluetooth operation underwent evaluation at varying distances up to 10 meters.

B. Wireless Communication Performance

Bluetooth communication reliability assessment involved transmitting 1000 sequential commands at varying distances and obstacle configurations. Results demonstrated 99.7% command delivery success within 5-meter line-of-sight range, with latency measured at 45-65ms from button press to motor response. This latency proves acceptable for human-operated control, with no perceptible lag in operator experience. Testing at extended range (8-10 meters) showed degraded performance with 8% command loss, primarily during periods of signal obstruction by metal structures.

WiFi video streaming evaluation measured frame rate stability and latency through network analysis tools. The system achieved consistent 15-18 fps at VGA resolution with frame-to-display latency measured at 180-220ms using VLC media player on various client devices. Network bandwidth consumption averaged 1.2 Mbps, well within typical institutional WiFi capacity. Image quality proved sufficient for navigation and object identification purposes, though lighting conditions significantly impact OV2640 camera performance with notable degradation in low-light scenarios.

C. Manipulator Positioning and Payload Assessment

Positioning accuracy evaluation employed optical tracking of arm end-effector position through repeated movement sequences. Testing involved commanding the arm to predefined positions and measuring actual achieved positions using calibrated reference markers. Results indicated positioning repeatability within $\pm 2^{\circ}$ across all joints, with slightly degraded performance ($\pm 3^{\circ}$) for the shoulder and elbow joints under maximum payload conditions. This accuracy suffices for intended demonstration and education applications, though would require improvement for precision manipulation tasks.

Payload capacity testing determined maximum grippable object weight before observable positioning error or servo stalling. The gripper successfully manipulated objects up to 100g weight with stable positioning. Heavier objects (150g+) caused noticeable servo strain with increased positioning error and elevated current draw. The base rotation joint demonstrated highest torque capacity supporting full arm extension

under 100g payload, while the wrist joint exhibited limitations at extreme angles. Continuous operation testing confirmed no thermal issues during 30-minute continuous manipulation sessions.

D. Mobile Platform Manoeuvrability Assessment

Mobility evaluation tested platform performance across various surface types including smooth tile, low pile carpet, and outdoor concrete surfaces. The tank configuration demonstrated superior traction compared to wheeled alternatives, successfully negotiating 15° inclines and small obstacles (up to 2cm height). Turning radius measurements indicated in-place rotation capability, essential for confined space operation. Maximum speed measured approximately 0.3 m/s, appropriate for controlled operation with live video monitoring. Battery testing under continuous operation yielded 2.5 hour runtime, matching design specifications. The tracked design exhibited expected limitations on loose surfaces (gravel, sand) where track slippage occurred.

VI. Discussion and Technical Contributions

A. Key Technical Innovations

This research demonstrates several significant technical contributions to accessible robotics. The distributed microcontroller architecture effectively addresses computational limitations through intelligent task partitioning, enabling capabilities typically requiring more powerful (and expensive) single-board computers like Raspberry Pi. The lightweight serial communication protocol between controllers minimises latency while maintaining architectural flexibility for future enhancements. Integration of WiFi video streaming with Bluetooth control within embedded hardware constraints represents practical IoT implementation relevant to numerous robotic applications.

B. Cost-Effectiveness Analysis

Total component cost for the complete system approximates $120 USD, representing substantial savings compared to commercial alternatives with equivalent capabilities typically exceeding $1000. This cost advantage enables deployment in educational contexts and developing regions where budget constraints severely limit access to robotic technologies. The modular architecture permits incremental assembly, allowing institutions to implement basic mobility or manipulation independently before full system integration. Open source software implementation eliminates licensing costs and enables customisation for specific applications.

C. Application Domains and Deployment Scenarios

The ZETABOT platform addresses multiple application domains. In precision agriculture, the system enables remote crop monitoring through video inspection while manipulator facilitates sample collection or selective harvesting tasks. Educational institutions benefit from comprehensive robotics platform suitable for undergraduate projects spanning mechanical design, embedded programming, wireless communication, and control systems.

Research applications include environmental monitoring in hazardous locations, with the system's low cost making multiple unit deployment economically feasible for sensor network applications.

Industrial inspection scenarios represent additional deployment opportunity, with the mobile platform navigating confined spaces while streaming live video and manipulating inspection instruments. The architecture's modularity supports customisation, such as alternative end-effector designs for specific manipulation requirements or sensor package integration for specialised monitoring applications. The solar charging integration particularly benefits extended deployment scenarios where regular battery servicing proves impractical.

D. Current Limitations and Areas for Improvement

Several limitations warrant acknowledgment and suggest directions for future enhancement. The video streaming implementation currently requires stable WiFi infrastructure, limiting autonomous deployment capability. Future iterations should explore direct ESP32-to-mobile WiFi connection enabling operation independent of existing network infrastructure. The manipulator control scheme employs open-loop positioning without feedback verification, potentially accumulating positioning errors during extended operation. Integration of low-cost potentiometer feedback or visual servoing capabilities would address this limitation. The mobile platform operates at fixed speed without obstacle detection, requiring constant operator attention. Ultrasonic or infrared proximity sensing integration would enable collision avoidance and semi autonomous navigation capabilities.

VII. Conclusions and Future Research Directions

This research successfully demonstrates that sophisticated mobile robotic capabilities including wireless vision systems, articulated manipulation, and intuitive control interfaces can be achieved within substantial budget constraints through innovative architectural design and careful component selection. The ZETABOT system validates distributed microcontroller approaches as viable alternatives to more expensive single board computer platforms for applications prioritising cost-effectiveness without requiring full computational autonomy.

Experimental validation confirms system performance meeting design objectives across communication reliability, manipulation accuracy, and operational endurance metrics. The achieved specifications prove adequate for educational robotics, agricultural monitoring, and light industrial inspection applications. The open-source implementation philosophy maximises accessibility and enables community driven enhancement, fostering collaborative development of accessible robotics technologies.

Future research directions include autonomous navigation integration through sensor fusion of ultrasonic, infrared, and vision systems with simultaneous localisation and mapping (SLAM) algorithms. Machine learning implementation for

object recognition and manipulation planning represents significant capability enhancement feasible within ESP32 computational constraints through optimised TensorFlow Lite models. Multi-robot coordination for collaborative tasks presents interesting research opportunities leveraging the system's wireless capabilities. Enhanced manipulator designs incorporating additional degrees of freedom or specialised end-effectors would expand application versatility. Power management optimisation through dynamic voltage scaling and intelligent sleep modes could substantially extend operational duration, particularly critical for field deployment scenarios.

REFERENCES

[1] Acieta. (n.d.). Robotics in manufacturing. https://www.acieta.com/why-robotic-automation/robotics-manufacturing/

[2] Arduino development boards. (n.d.). https://eu.mouser.com/applications/open-source-arduino/

[3] M. Banzi and M. Shiloh, *Getting started with Arduino: The open source electronics prototyping platform*. Maker Media, Inc., 2014.

[4] T. Bräunl, *Embedded robotics: Mobile robot design and applications with embedded systems*, 3rd ed. Springer, 2008.

[5] J. J. Craig, *Introduction to robotics: Mechanics and control*, 4th ed. Pearson Education, 2017.

[6] DC motor direction control using Arduino Uno with L298 driver. (n.d.). https://www.ukessays.com/essays/engineering/dc-motor-direction-control-using-arduino-uno-with-l298-driver.php

[7] DroneBot Workshop. (n.d.). ESP32-CAM – Getting started & solving common problems. https://dronebotworkshop.com/esp32-cam-intro/

[8] G. Dudek and M. Jenkin, *Computational principles of mobile robotics*, 2nd ed. Cambridge University Press, 2010.

[9] Electronics for You. (2021, July 17). 16x2 LCD pinout diagram — Interfacing 16x2 LCD with Arduino. https://www.electronicsforu.com/technology-trends/learn-electronics/16x2-lcd-pinout-diagram

[10] Espressif Systems. (2021). *ESP32 technical reference manual* (Version 4.5). https://www.espressif.com/documentation

[11] ESPBoards. (n.d.). MG996R servo pinout, wiring, and more. https://www.espboards.dev/sensors/mg996r/

[12] Everything you need to know about robotic arms. (n.d.). https://ie.rs-online.com/web/generalDisplay.html?id=ideas-and-advice/robotic-arms-guide

[13] Genesis Systems. (n.d.). 6 common applications for robots in automotive manufacturing. https://www.genesis-systems.com/blog/robots-automotive-manufacturing-top-6-applications

[14] Getting started with ESP32-CAM board & video streaming over WiFi. (n.d.). *How To Electronics.* https://how2electronics.com/getting-started-with-esp32-cam-board-video-streaming-over-wifi/

[15] U. Jahn, C. Wolff, and P. Schulz, "Concepts of a modular system architecture for distributed robotic systems," *Computers*, vol. 8, no. 1, 2019. https://doi.org/10.3390/computers8010025

[16] R. N. Jazar, *Theory of applied robotics: Kinematics, dynamics, and control*, 2nd ed. Springer, 2010.

[17] A. Junaidi, A. Yudhana, and S. Sunardi, "Internet of Things Car Prototype Based on ESP32-CAM," *IJEIS (Indonesian Journal of Electronics and Instrumentation Systems)*, 2021.

[18] Kodular. (2021, May 30). Everybody Wiki bios & wiki. https://en.everybodywiki.com/Kodular

[19] R. K. Kodali and S. Soratkal, "MQTT based voice-controlled home automation system using ESP32," in *2020 3rd International Conference on Intelligent Sustainable Systems (ICISS)*, 2020.

[20] R. K. Kodali, G. Swamy, and B. Lakshmi, "An implementation of IoT for healthcare," in *2016 IEEE Recent Advances in Intelligent Computational Systems (RAICS)*, 2016.

[21] D. Kolpashchikov, O. Gerget, and R. Meshcheryakov, "Raspberry Pi, Arduino, and LEGO Mindstorms NXT in robotics education: Comparison and evaluation," in *Proceedings of the Computational Methods in Systems and Software*. Springer, 2019.

[22] T. R. Kurfess, Ed., *Robotics and automation handbook*. CRC Press, 2018.

[23] L298N motor driver board – Geeetech Wiki. (n.d.). https://www.geeetech.com/wiki/index.php/L298N_Motor_Driver_Board

[24] A. Maier, A. Sharp, and Y. Vagapov, "Comparative analysis and practical implementation of the ESP32 microcontroller module for the Internet of Things," in *2017 Internet Technologies and Applications*, 2017.

[25] Motor driver module-L298N – Wiki. (n.d.). http://wiki.sunfounder.cc/index.php?title=Motor_Driver_Module-L298N

[26] NASA. (2009, November 9). What is robotics? https://www.nasa.gov/audience/forstudents/k-4/stories/nasa-knows/what_is_robotics_k4.html

[27] R. P. Paul, *Robot manipulators: Mathematics, programming, and control*. MIT Press, 1981.

[28] Robotics: A brief history. (n.d.). Stanford University. https://cs.stanford.edu/people/eroberts/courses/soco/projects/1998-99/robotics/history.html

[29] Robots in the lab. (2012, May 10). *Automate.* https://www.automate.org/industry-insights/robots-in-the-lab

[30] B. Siciliano and O. Khatib, *Springer handbook of robotics*. Springer International Publishing, 2016.

[31] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to autonomous mobile robots*, 2nd ed. MIT Press, 2011.

[32] S. G. Tzafestas, *Introduction to mobile robot control*. Elsevier, 2013.

[33] UK Essays. (n.d.). A problem statement of robotics technologies information technology essay. https://www.ukessays.com/essays/information-technology/a-problem-statement-of-robotics-technologies-information-technology-essay.php?vref=1

APPENDICES

GitHub Repository (Source Code and Project Files): https://github.com/Iyanuoluwa007/zetabot.git